Lab:
Intro to Python 2

Hakyung Sung
University of Hawaii at Manoa

September 2, 2026

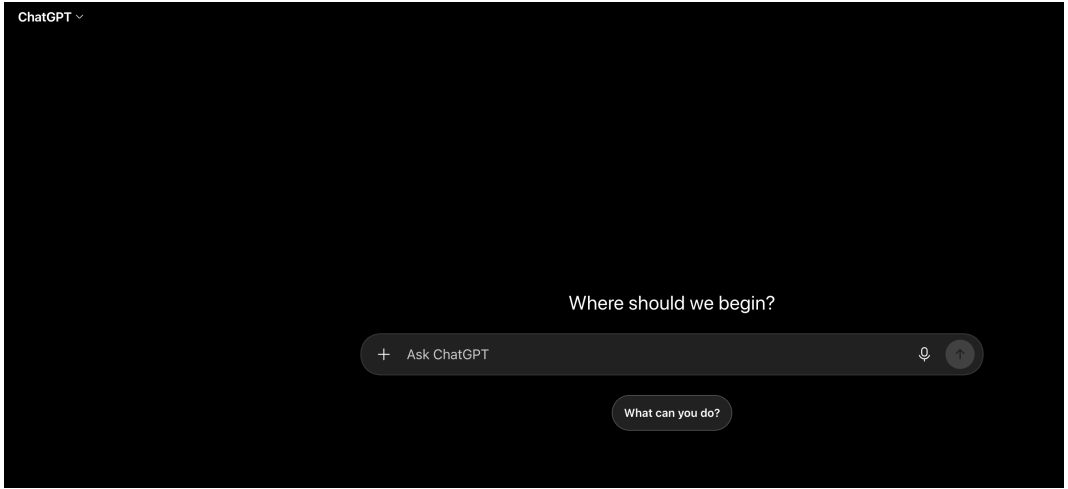
Topics to cover

- Basic structure of a NLP tool
- (Review) Basics I
- Basics II, III

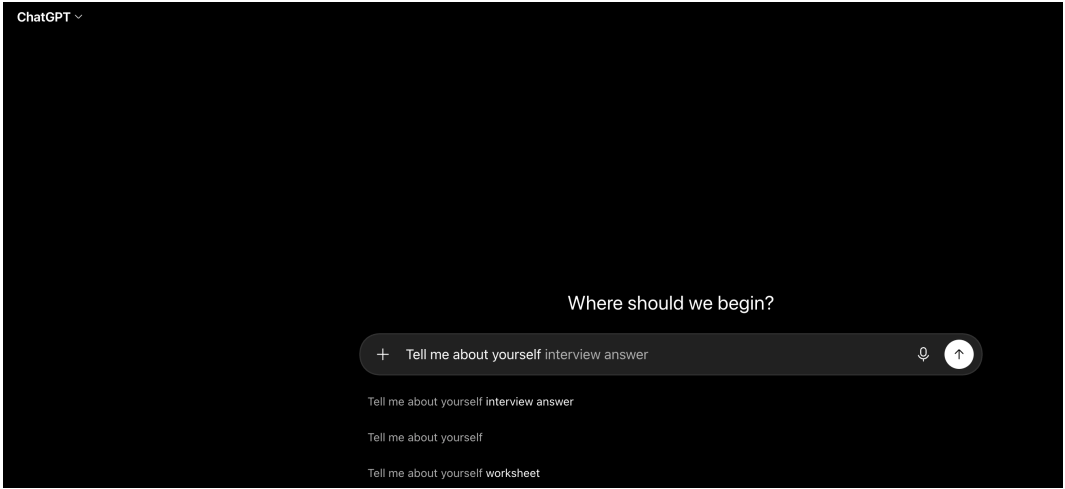
Basic structure of a NLP tool

NLP tools share a similar logic.

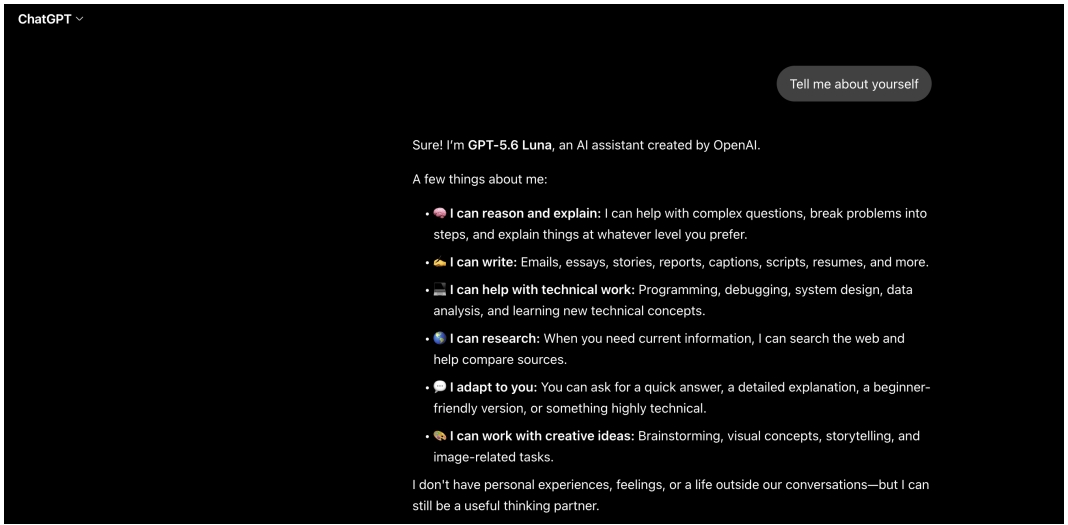
ChatGPT

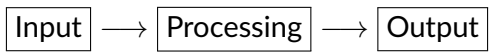


ChatGPT


ChatGPT
Where should we begin?
Tell me about yourself interview answer
Tell me about yourself interview answer
Tell me about yourself
Tell me about yourself worksheet

ChatGPT

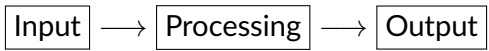

ChatGPT
Tell me about yourself
Sure! I'm GPT-5.6 Luna, an AI assistant created by OpenAI.
A few things about me:
I can reason and explain: I can help with complex questions, break problems into steps, and explain things at whatever level you prefer.
I can write: Emails, essays, stories, reports, captions, scripts, resumes, and more.
I can help with technical work: Programming, debugging, system design, data analysis, and learning new technical concepts.
I can research: When you need current information, I can search the web and help compare sources.
I adapt to you: You can ask for a quick answer, a detailed explanation, a beginner-friendly version, or something highly technical.
I can work with creative ideas: Brainstorming, visual concepts, storytelling, and image-related tasks.
I don't have personal experiences, feelings, or a life outside our conversations—but I can still be a useful thinking partner.



- What do I give the tool? **a sentence**
- What do I want back? **some descriptions of the information**

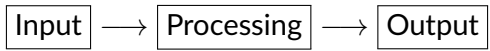
NLP tool starts with input and output

At the most basic level, an NLP tool does this:



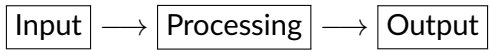
- **Input**: What do I give the tool?
- **Output**: What exactly do I want back?
- **Processing**: What needs to happen in between?


Whirlpool
12:20



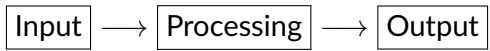
- **Input**: What do I give the tool? frozen rice
- **Output**: What exactly do I want back? eatable rice
- **Processing**: What needs to happen in between? A(n) algorithm/mechanism/magic (?) that transforms the input into the output


LG
Mode
Fan
Light
Timer
Swing
LG



- **Input**: What do I give the tool? stuffy hot air in my room
- **Output**: What exactly do I want back? nice cool air
- **Processing**: What needs to happen in between? A(n) algorithm/mechanism/magic (?) that transforms the input into the output

Back to a text example



- **Input**: What do I give the tool? “I like corpus linguistics.”
- **Output**: What exactly do I want back? “I LIKE CORPUS LINGUISTICS.”
- **Processing**: What needs to happen in between?

What we learned in Python

"I like corpus linguistics." ⟶ "I LIKE CORPUS LINGUISTICS."

- **Values**: the actual pieces of data

```
"I like corpus linguistics."
```

- **Variables**: names that store values

```
text = "I like corpus linguistics."
```

- **Methods**: operations attached to an object

```
text.upper()
```

- **Functions**: reusable processing that takes input and returns output

```
make_uppercase(text)
```

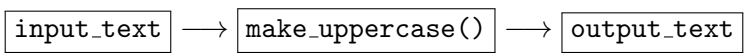
Putting everything together

```
def make_uppercase(text):
    return text.upper()

input_text = "I like corpus linguistics."

output_text = make_uppercase(input_text)

print(output_text)
```



Basics I

Basics I.1. Values: Different kinds of data

```
text = "apple"
number = 10
decimal = 2.5
is_ready = True
empty = None
```

- **strings**
- integers
- floats
- booleans
- `None`

Basics I.1. Values: Type matters

```
"9" + "3"   # "93"

9 + 3       # 12

"9" + 3     # error
```

The type of a value determines what operations and methods are available.

Basics I.1. Values: Integers and floats

```
count = 7
price = 3.5

count + 1       # 8
price * 2       # 7.0

7 / 2           # 3.5
7 // 2          # 3
```

Numeric types represent different kinds of numbers and support arithmetic operations.

Basics I.1. Values: Booleans

```
5 > 3       # ?

2 == 4       # ?
```

Basics I.5. Exercise 1

`Exercise1_Firstname_Lastname.py`

Complete the exercises from Basics I.

Your task

- Create one Python file
- Complete the assigned exercises
- Run your code and check the output
- Save the file using the required filename

Filename: `Exercise1_Firstname_Lastname.py`

Basics I.5. Exercise 1: Using VS Code Chat

- Several ways to use Chat:
  - ask what a piece of code means;
  - ask why you are getting an error;
  - ask how to modify your code;
  - **check whether your code does what you intended**.

- Try to use at least one chat session for each exercise.

Basics I.5. Exercise 1: Save your chat record

- After completing the exercise, export your VS Code Chat session.
- Save the exported chat record as:

`Exercise1_Firstname_Lastname_chat.json`

- Submit:
  - `Exercise1_Firstname_Lastname.py` (1 point)
  - `Exercise1_Firstname_Lastname_chat.json` (1 point)

Basics I.5. Exercise 1 (Example)

Class demo

Basics II, III

Basics II.1. Strings

```
s = "sample"

s[0]      # "s"
s[-1]     # "e"

s[0:2]    # "sa"
s[1:]     # "ample"

"sam" in s   # True
```

A string is a **sequence of characters**.

- `[ ]`: access part of a string
- indexing starts at `0`
- `[start:end]`: slicing
- `in`: check membership

Basics II.2. Lists

```
langs = ["Python", "R", "Java"]

langs[0]        # "Python"
langs[1:]       # ["R", "Java"]

langs.append("C++")

langs[0] = "Julia"
```

A list is an **ordered collection of values**.

- items can be accessed by index
- lists can contain multiple values
- lists are **mutable**
- `.append()` adds an item
- `.remove()` removes an item

Basics II.3. Conditional Statements

```
x = 2

if x < 2:
    print("less")

elif x == 2:
    print("equal")

else:
    print("greater")
```

Conditionals let the program make a **decision**.

- `if`: check a condition
- `elif`: check another condition
- `else`: otherwise

Basics II.4. Loops

```
words = ["read", "code", "repeat"]

for w in words:
    print(w)


count = 0

while count < 3:
    print(count)
    count += 1
```

Loops let us **repeat an operation**.

- `for`: repeat for each item
- `while`: repeat while a condition is true

Basics III.1. Tuples

```
t = (1, 2, 3, 4)

t[0]        # 1
t[-1]       # 4
t[1:3]      # (2, 3)

x, y, *rest = t
```

A tuple is similar to a list, but it is **immutable**.

- ordered sequence
- supports indexing and slicing
- **cannot be changed in place**
- useful for fixed groups of values

Basics III.2. Dictionaries

```
freq = {
    "the": 69033,
    "of": 36998,
    "and": 30157
}

freq["the"]       # 69033
freq["to"] = 21892

freq.keys()
freq.values()
freq.items()
```

A dictionary stores **key–value pairs**.

- each key identifies a value
- access values using a key
- keys must be unique
- very common for storing linguistic information

Basics III.3. Functions

```
def greet(name):
    return "Hello, " + name

message = greet("Alice")


def divide(a, b):
    return a / b
```

Functions package **reusable processing**.

- parameters receive input
- code inside performs processing
- `return` sends output back

Basics III.4. Classes (advanced)

```
class Text:
    def __init__(self, content):
        self.content = content

    def make_lowercase(self):
        return self.content.lower()

text = Text("HELLO")
text.make_lowercase()
```

A class groups **data and behavior** together.

- class: blueprint
- object: an instance of the class
- attributes: data
- methods: operations

Basics III.5. Working with Files

```
# Write
with open("fruits.txt", "w") as f:
    f.write("apple\n")

# Read
with open("fruits.txt", "r") as f:
    text = f.read()
```

Programs often need to read or save data.

- `"r"`: read
- `"w"`: write
- `open()`: open a file
- `with`: close it automatically

Different Ways to Store Data

- `.txt` — plain text
- `.csv` — rows and columns
- `.json` — structured key–value data

Basics II, III. Exercises

```
Exercise2_Firstname_Lastname.py (1 point)
Exercise2_Firstname_Lastname_chat.json (1 point)
Exercise3_Firstname_Lastname.py (1 point)
Exercise3_Firstname_Lastname_chat.json (1 point)
```

Your task

- Create one Python file
- Complete the assigned exercises
- Run your code and check the output
- Save the file using the required filename

Exercise 4: Find the Longest Word in Each Line

Suppose you are given a text file called `sentences.txt`.

Example file:

```
I like corpus linguistics
Python is useful for text analysis
Natural language processing is fascinating
```

For each line, find and print the **longest word**.

Desired output:

```
linguistics
analysis
fascinating
```

▸ hint!

Preview: Assignment 1 (Deadline: 9/11)

Each exercise is worth **2 points**. Total: **10 points**.

- **Exercise 1**: Submit your `.py` and .json
- **Exercise 2**: Submit your `.py` and .json
- **Exercise 3**: Submit your `.py` and .json
- **Exercise 4**: Submit your `.py` and .json
- **Exercise 5**: Complete *A Survey of Corpora* (next week)

Appendix

Decompose the problem

Think about what needs to happen for **each line**:

1. Split the line into words.
2. Start with one word as the current longest word.
3. Look at each word one at a time.
4. Compare its length with the current longest word.
5. If it is longer, replace the current longest word.
6. Print the final result.

file → lines → words → compare lengths → longest word